



Week 8

- **Design Pattern**
 - **Singleton**
 - **Factory**
 - **Builder**
 - **Strategy**
 - **Adapter**
- **Agile Planning - Burndown Chart**



Design Principle vs Design Pattern

Design Principle

- Design principles provide high level guidelines to design better software applications.
- They do not provide implementation guidelines and are not bound to any programming

Design Pattern

- Design Pattern provides low-level solutions related to implementation, of commonly occurring object-oriented problems
- Design pattern suggests a specific implementation for the specific object-oriented programming problem

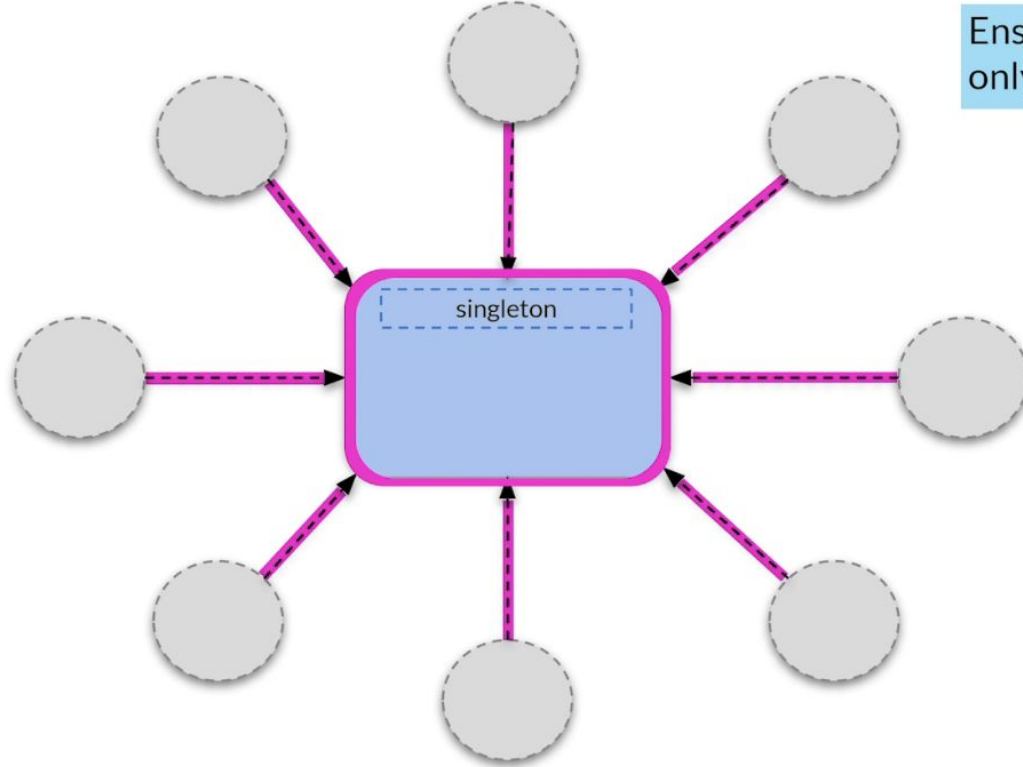
Design Pattern - Singleton

Singleton is a **creational design pattern** that ensures that a class has only one instance and provides an easy global access to that instance.

The pattern does not stipulate what to do with a singleton.

The main tenets of this pattern:

- Ensure that the class has only a single instance.
- Provide easy global access to this instance.
- Control how it is instantiated.
- Any **critical region** must be entered serially.

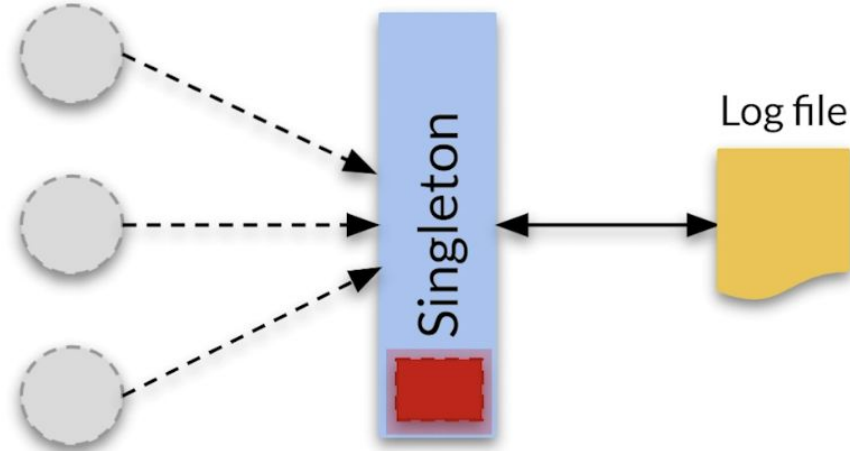


Ensure that the class has only a single instance.

What can be done inside Singleton

- Loggers
- Caching
- Threadpool
- Database connection
- Configuration access

logging



Contd.

This pattern is one of the famous **Gang of Four** patterns, and its motivation is stated as follows.

“It is important for some classes to have exactly one instance. Although there can be many printers in a system, there should only be one printer spooler. There should be only one file system and one window manager.”

This pattern is often used with following GOF patterns

- Abstract factory
- Builder
- Prototype
- Facade
- State



When to Use or Not

When to use ?

- When you want to control access to a shared resource?

When not to use.

- Use the singleton pattern with restraint and do not let it degenerate into just global access for everything. Global access hides dependencies and might make it harder to read such code. So make sure that you have a good reason to use this pattern.
- The main question you should ask yourself as to whether you're going to use or not use the singleton pattern is do you violate the SRP, the single responsibility principle? If yes, then you should reconsider using it.



Design Consideration

- When designing a Singleton ,consider **lazy construction**, which means that the class instance should only be created when it is first Needed. In some cases we might consider **eager loading** if for example, we need the singleton to be always ready and load it fast. The main difference here is very much with memory.
- **Threat safety** - You need to consider that specifically in languages that allow multithreaded access because you need to ensure that access is properly controlled and locked so that the state of the singleton, if it has one, is always deterministic.

Factory Method Pattern

- How do we create object ? Normally ,it's using constructor
- What's incorrect with this approach ?

```
Rectangle rect = Rectangle();
```

Scenario:

Assume you are creating a space shooter game



Asteroid



Space Invader

Contd.

- Different types of bullets that the user could use
 - But you do not know which bullets the user will want to use.
 - So how do you create them?
- Additionally, you might in the future have new bullet types, so you might have types that you haven't even thought of yet.

```
FastBullet fb = FastBullet();  
SlowBullet sb = SlowBullet();  
SplashBullet spb = SplashBullet();  
? ob = ?();
```

Problem ??

Contd.

The factory method is a creational design pattern that lets us abstract out the creation logic of specific class instances. So it provides a mechanism for creation of objects without exposing the instantiation logic to the client.

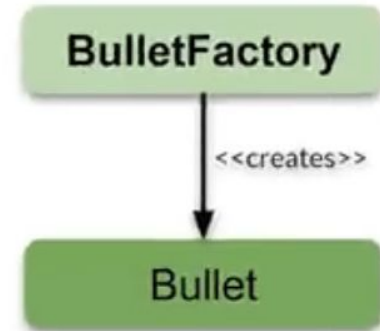
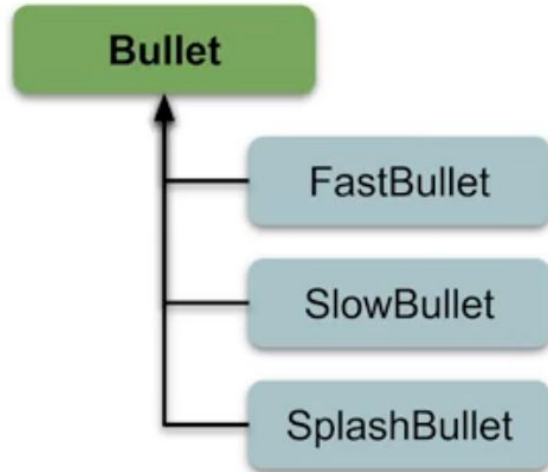
There are two main and crucial points in understanding this pattern.

- Objects are created by calling a factory method instead of calling a constructor.
- Objects are created through an abstraction, not a concretion..

Factory Method: replace **direct** object construction calls **with** calls to a special factory method.

```
FastBullet fb = FastBullet();  
SlowBullet sb = SlowBullet();  
SplashBullet spb = SplashBullet();  
? ob = ?();
```

Contd.





Contd.

```
Bullet myBullet001 = BulletFactory.create(Bullet.FAST_BULLET);  
Bullet myBullet002 = BulletFactory.create(Bullet.SPLASH_BULLET);
```

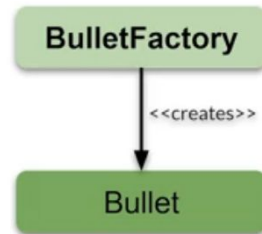
- The main advantage is that the caller does not need to know how the factory creates the bullet. The caller only needs to know **HOW to call** the factory and **WITH what** initialization data.
- We could even make this more generic by using some type of **context** data to initialize the instance.

```
BulletBuildContext context = // some initialization data  
Bullet myBullet002 = BulletFactory.create(context);
```

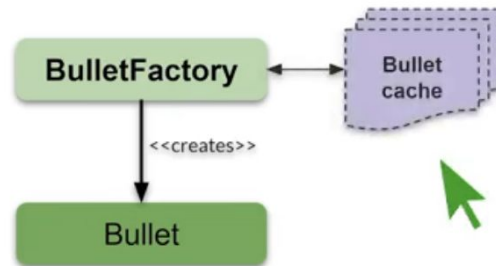
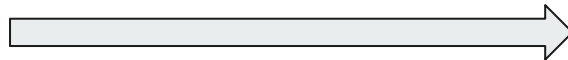
Advantages

One example advantage of using a factory method pattern is that we could cache objects in the factory

Factor without cache:



Factory with cache but it's hidden





What exactly is Factory Method Pattern

This is another famous gang of four pattern and its motivation is stated as follows.

Define an interface. For creating an object, but let subclasses decide which class to instantiate.

Factory method lets a class defer instantiation to the subclasses.

This pattern is often used with the following Gang of four patterns.

- Strategy and iterator patterns are often used with factory method pattern to let collection subclasses return different types of iterators that are compatible with the collections.
- Factory method pattern can also be used in the object pull pattern when creating cached objects of different subtypes.

When to Use & Not Use FMP

When a caller can't anticipate the types of object it must create?

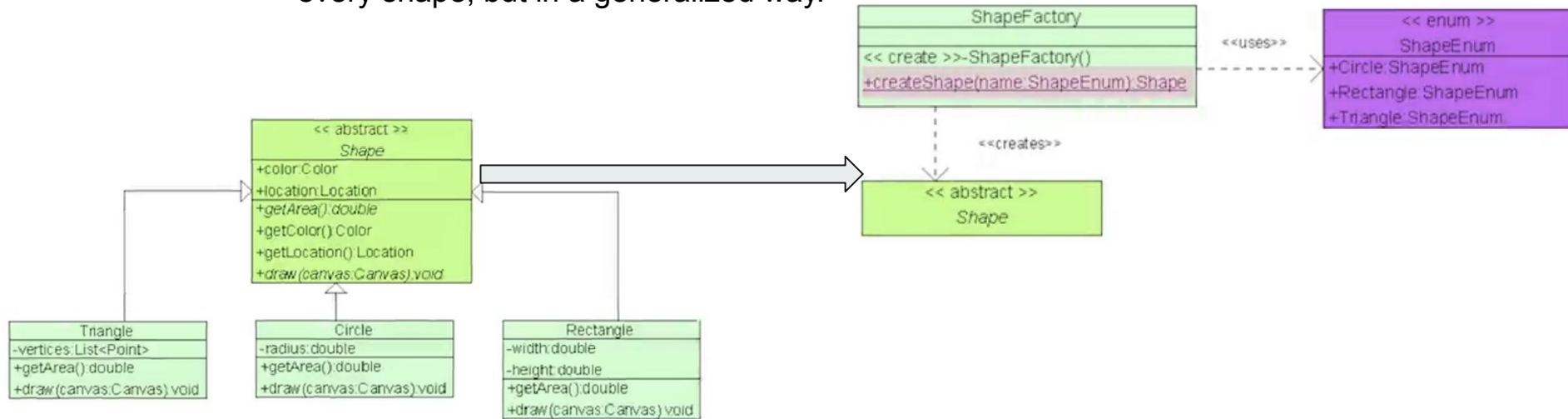
- For ex. coding a mobile game.
 - The user can choose any weapon or a spaceship, but you do not know which one they will use.
 - Another example is new types of spaceships or weapons can be added to the game in the future without any changes to the client code.
- When you have many objects of a common type, like, for example, a shape with subclasses such as Rectangle, Circle Triangle, it's a perfect opening for a factory method pattern

When not to use

- There is no specific restrictions as to when **NOT** to use it.

Design Considerations

- When you have many objects of a common type like for example a *Shape*
 - Make all such object types (like for example *IShape*), follow the same interface or extend the same abstract class. This interface should then declare methods that make sense in every shape, but in a generalized way.





Contd.

- Create a factory with a static creation method which always returns the same common interface reference, for example, like an *Ishape* or like what we had with the *bullet*.
- Let the factory know which instance you want through some sort of a constant ID for the specific object type.
 - This can be accomplished through maybe an enum or a list of constants which could be integers or strings.

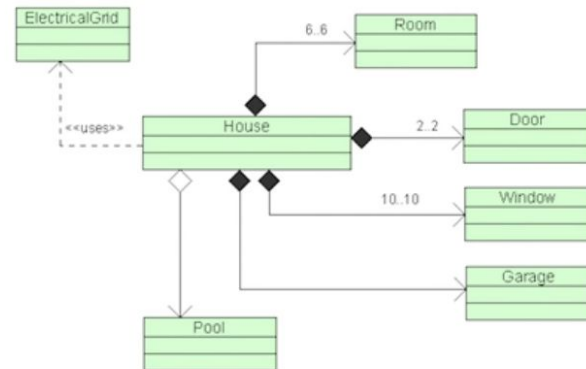
Builder Designer Pattern

Builder is a creational design pattern that is used to encapsulate **reusable** logic of building **complex** objects

Complex Objects - Those are objects that are made up of many distinct parts.

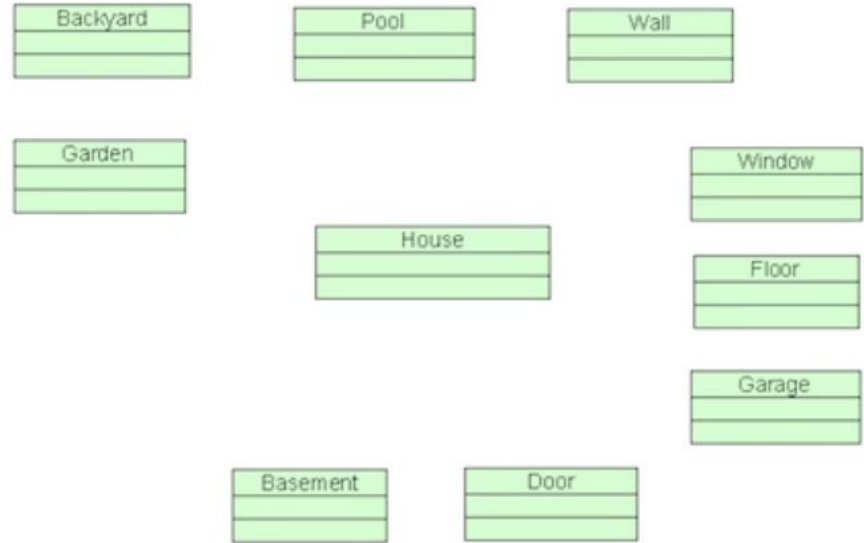
The main tenets of this pattern

- Separate the construction of a complex object from its representation
- Design it in a manner so that the same construction process can create different representations.



Imagine that you're coding the process of building a house for a video game with the following attributes.

- Walls
- Windows
- Floor
- Garage
- Swimming Pool
- Door
- Backyard
- Garden
- Basement
- etc.



```
House(10, 10, 2, 1, 0, 5, false, false, false, ...)
```

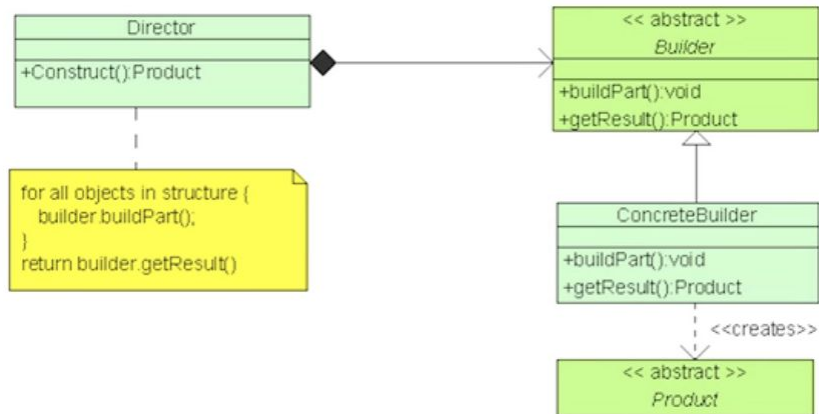
Contd.

- We want to extract the object construction away from the class representation.
- In other words, we do not let the object create itself to the user. We want to abstract the creation process.

How to Do it

- By using Builder Pattern
- It hides the process of building complex objects.
- A builder is an object that is able to construct other objects.

it constructs and assembles different object parts to build the result objects.

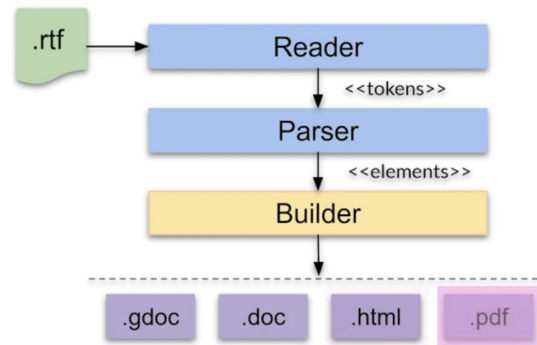
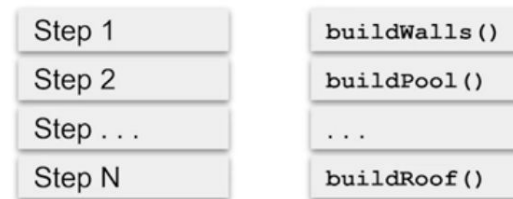


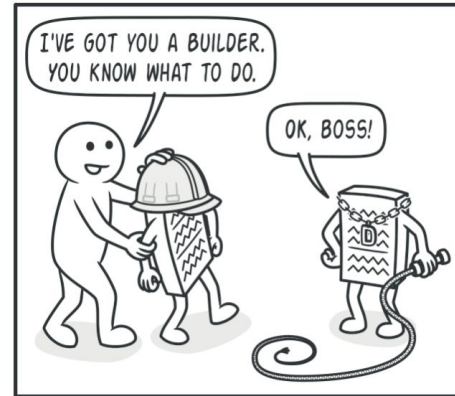
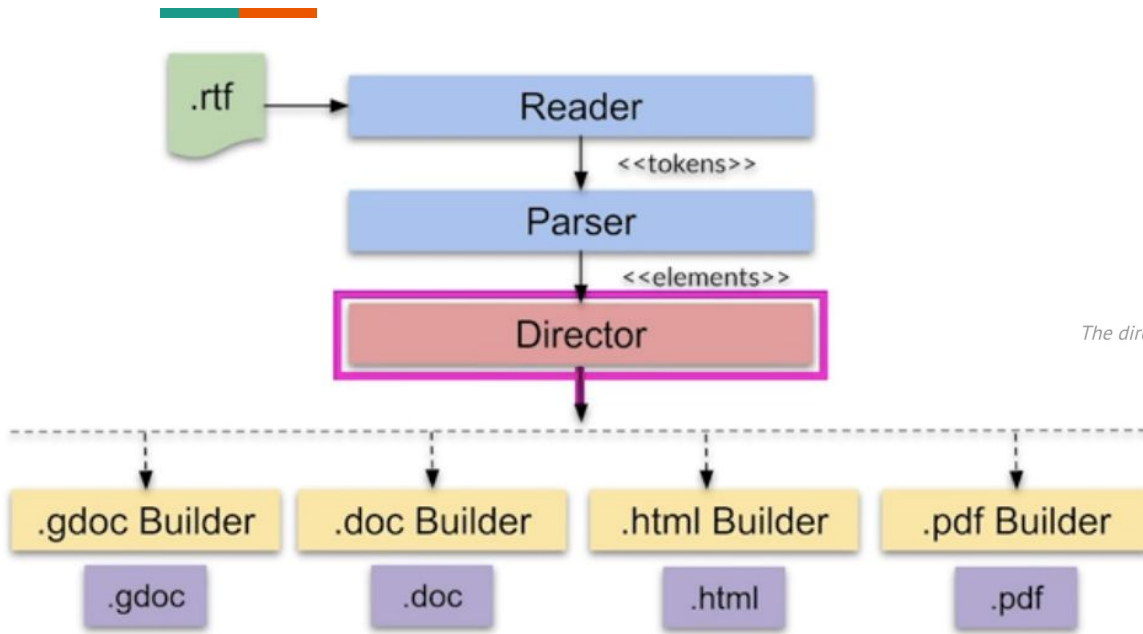
Contd.

- A builder organizes object construction into a set of processing steps
- To create an object. The builder is instructed to execute a of these steps.

Consider another example - a document in your hard drive

```
for each element parsed
  switch on element.type
    case: TITLE
      builder.createTitle(element)
    case PAGE
      builder.createPage(element)
    case AUTHOR
      builder.createAuthor(element)
```





The director knows which building steps to execute to get a working product.



Contd.

This is an important ***Gang of four*** pattern and its motivation is stated as follows.

Separate the construction of a complex object from its representation so that the same construction process can create different representations.

The pattern is often used with the following gang of four patterns:

- Composite pattern when you are creating tree like aggregation of objects
- It will be often implemented as a singleton.



When to Use & Not Use it

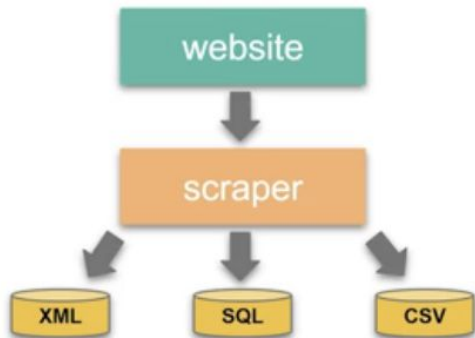
- When you have a complex class with many constructors because the builder pattern lets you build your objects in a step by step manner, you can skip some steps if they are not needed.
- When you have to build complex composite tree like objects
- When you want to build different representation of your complex object but still use the same general steps.

NOT to Use

- When there is no need to use this pattern for **simple objects**. At that point you can just use the factory method for those.

Strategy Design Pattern

1. Let's assume that you have a web bot (i.e. like a web crawler) and you want it to scrape some data from the internet.
2. You might be interested in data content such as XML files, SQL files, CSV files, perhaps JSON files, etc...



Here is simple pseudocode for our scraper:

```
if(protocol is 'xml')
    { extract xml data }
if(protocol is 'sql')
    { extract sql data }
if(protocol is 'csv')
    { extract csv data }
```

Code Bloat

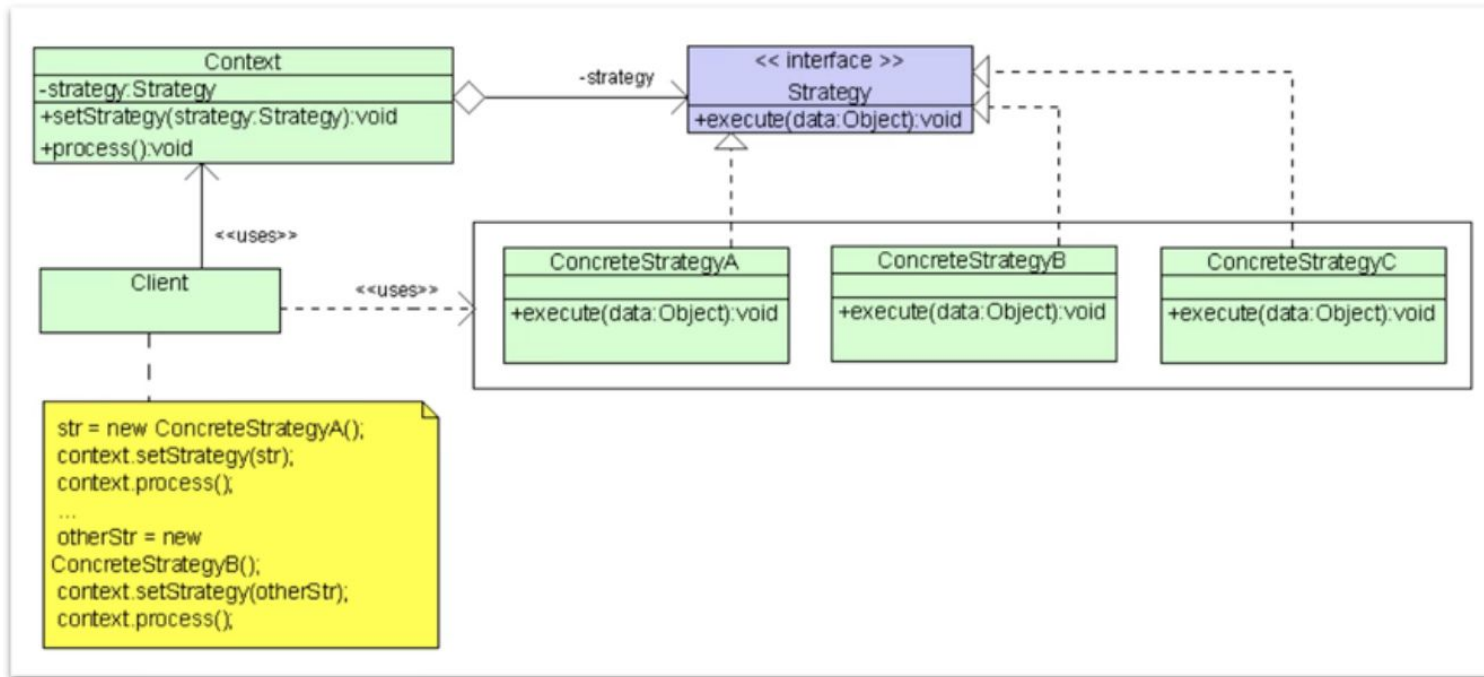


What's the problem ?

- The scraper class will grow uncontrollably with each new data protocol.
 - The extraction code is not reusable, it's only in the scraper.
 - The design violates the solid principles, specifically the open close principle, which states that we should extend the class, not modify it
-
- One aspect of the solution would be to abstract each protocol so the CSV and the XML etc. into its own class and then have the scraper call that specific class when needed.



- Defer the decision until runtime which algorithm to use.
- Expose the clients (scraper) , which in this case there's a scraper only to the interface of the algorithm
 - This will allow seamless substitution
- Use composition to loosely couple the handler with the parent decision making class.



The strategy pattern is the crowning jewel of all the Gang of four design patterns.



When to Use & Not Use

- Use this pattern to obstruct the business logic of class from its implementation details so that it can be plugged in.
- Use it when your class has a potential conditional statement that switches between different variants of the same algorithm.
- Use the strategy pattern when you want to use different variants or version of an algorithm within an object and need to have the ability to switch from one algorithm to another during runtime.

When Not To Use

- If you only have a few algorithms and they rarely change, it might be unnecessary to overcomplicate your code with new classes and interfaces.



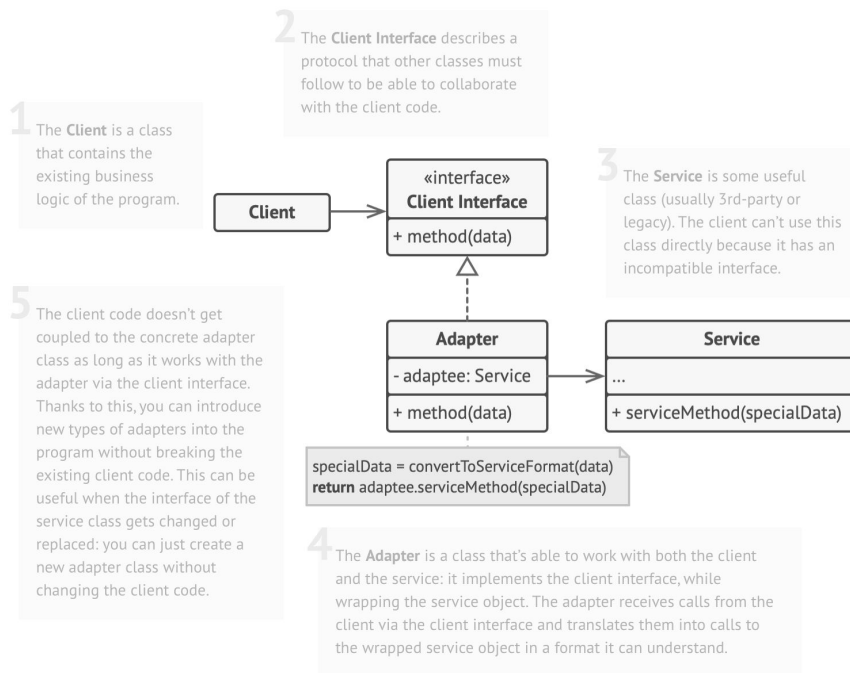
Adapter

Existing object that provide functionality that client needs, but client code can't use this object because it expects an object with different interface.

Make an existing object work with client by adapting the object to client's expected interface

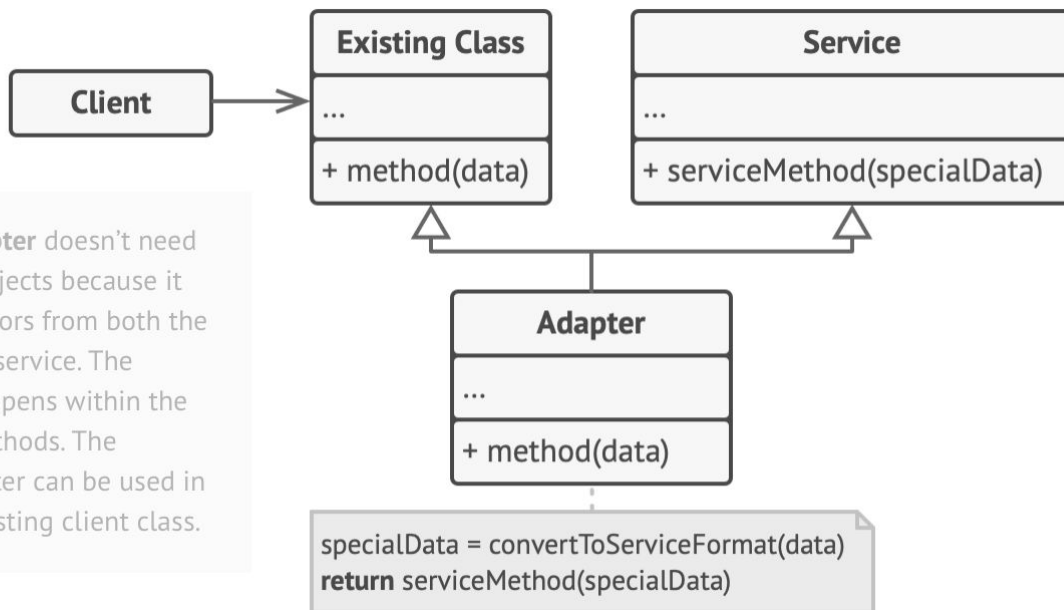
Also called as wrapper as it wraps existing object

Object Adapter



Class Adapter

1 The **Class Adapter** doesn't need to wrap any objects because it inherits behaviors from both the client and the service. The adaptation happens within the overridden methods. The resulting adapter can be used in place of an existing client class.





Agile Planning - Team Velocity

Team velocity, in short, is how many points of stories can the team, this project team, complete in a given sprint? So, ultimately, what's the average number of story points

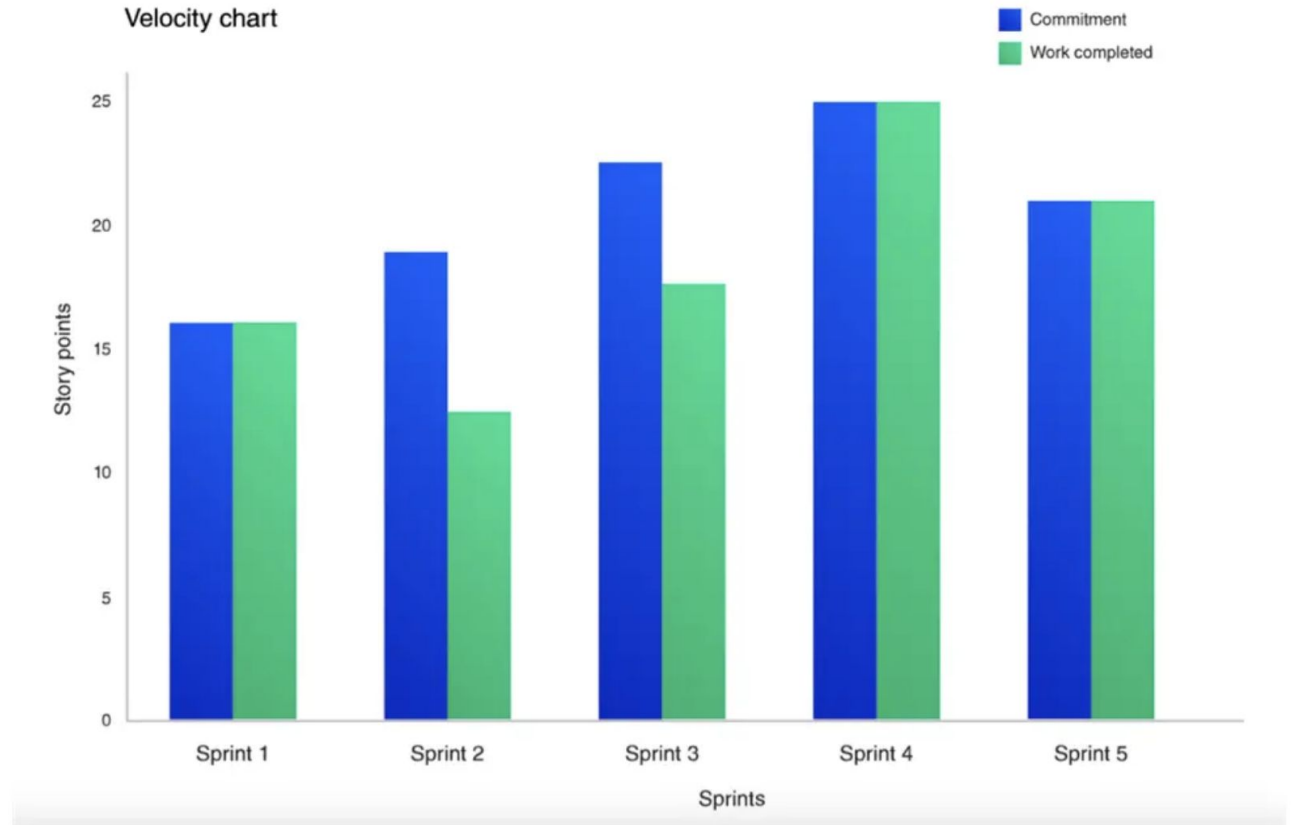
Sprint one. Your team has committed to eight user stories, and each story equals three story points. Let's say the team only completes four stories. You would achieve **12 story points** in total.

Sprint two. Your team has committed to 10 user stories, and each story equals five story points. Let's say your team completes seven stories. You would end up with **35 story points**.

Sprint three. Your team commits to nine user stories, and each story equals four story points. Let's say your team completes seven stories. You'd have **28 story points**.

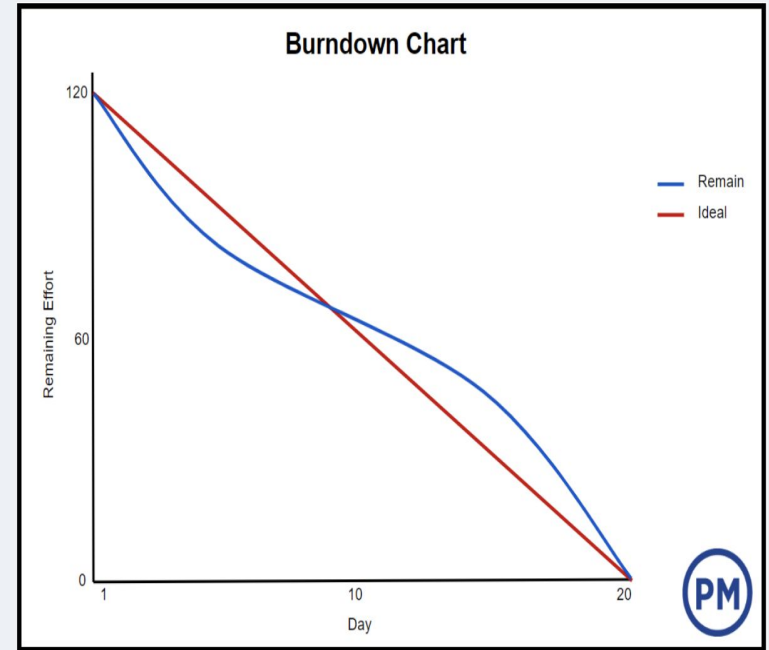


Contd.



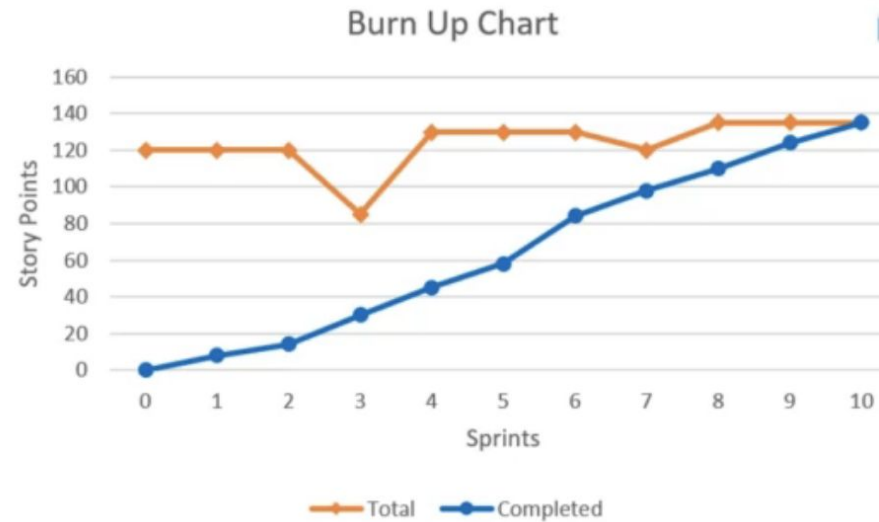
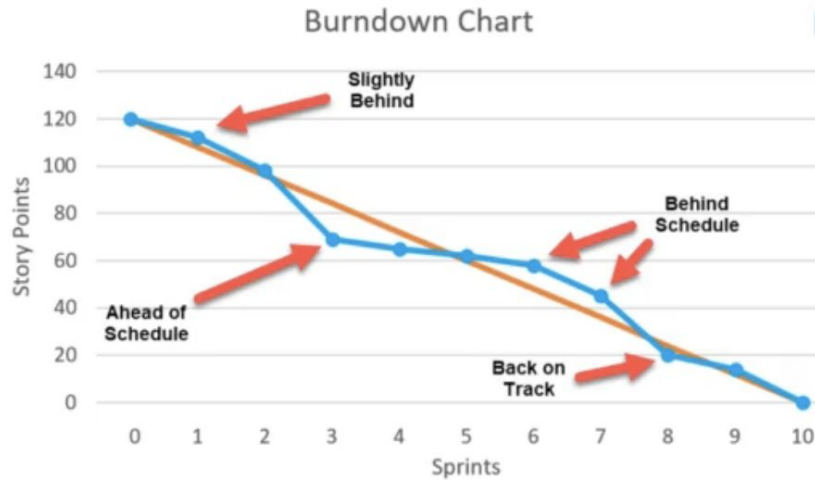
Burndown Chart

- A burndown chart is a project management chart that shows how quickly a team is working through a client user stories.
- There are two burndown chart variants: a sprint burndown and a product burndown. A sprint burndown is used for work remaining in the iteration while a product burndown illustrates the work remaining for the entire project
- A burndown chart has two axes, x and y. The horizontal axis represents time while the vertical axis displays user story points. The rightmost point of the chart indicates the start of a project or agile sprint while the leftmost point shows its end.



Burndown Chart Example

Burn Up chart





Thank you